

Prova Finale - Reti Logiche

Alberto Biffi 10677341 - 933268
Giovanni Mattia Codemo 10707329 - 934968

Anno Accademico 2021-2022

Contents

1	Requisiti del Progetto	2
1.1	Funzionamento Convolutore	2
1.2	Richieste di Progetto	2
2	Implementazione	3
2.1	Algoritmo	3
2.2	Macchina a Stati Finiti	5
2.3	Schema dell'Implementazione	7
3	Risultati sperimentali	7
3.1	Testing	7
3.2	Report di Sintesi	7
4	Note Conclusive	8

1 Requisiti del Progetto

Il progetto consiste nella progettazione di un modulo hardware che implementa un codificatore convoluzionale con tasso di trasmissione $\frac{1}{2}$. Il linguaggio di descrizione utilizzato è il VHDL, mentre il software è Vivado di Xilinx.

1.1 Funzionamento Convolutore

Un codificatore convoluzionale è un dispositivo che ricevendo in input una sequenza continua di N parole, trasmette in output una serie continua di M parole (nel nostro caso $M = 2 * N$). Esso viene impiegato nel settore delle telecomunicazioni per la rilevazione e correzione di eventuali errori di trasmissione e il suo funzionamento può essere riassunto dalla seguente macchina a stati finiti:

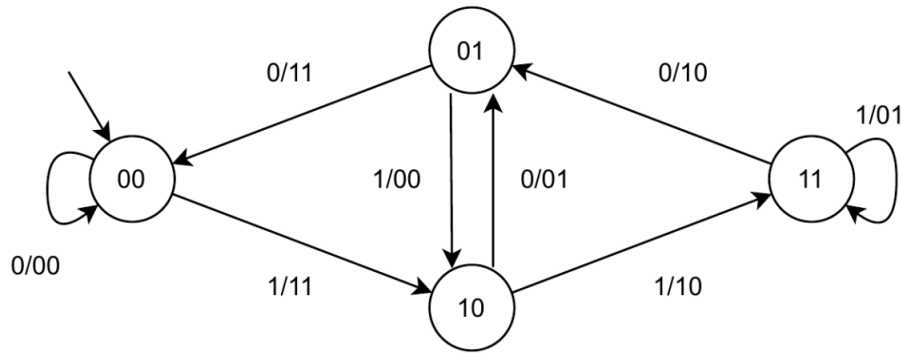


Figura 1: FSM Convolutore

1.2 Richieste di Progetto

Al componente viene richiesto di:

1. Accedere in lettura ad una memoria RAM fornita e leggere all'indirizzo 0 i byte da codificare
2. Partire dall'indirizzo successivo alla prima lettura a leggere i byte da codificare
3. Stampare a partire dall'indirizzo 1000, in sequenza, i byte codificati
4. Ripetere il processo fino alla fine di tutti i byte presenti in memoria

Terminata una computazione, al componente può essere richiesta la codifica di un altro stream in ingresso. Inoltre, in ogni momento della computazione, può essere dato un segnale di reset che riporta la macchina nello stato di partenza e la prepara per una nuova computazione.

2 Implementazione

2.1 Algoritmo

L'algoritmo scelto segue i seguenti passi:

1. Legge l'indirizzo 0 della RAM e salva nel registro "byteToRead" il numero totale di byte da codificare
2. Legge l'indirizzo successivo e salva il byte letto nel registro "w_bitStream"
3. Legge 4 bit del registro e salva l'output nel registro temporaneo "tmp"
4. Stampa il registro temporaneo a partire dall'indirizzo prefissato della RAM e se non ha letto tutti i bit di "w_bitStream" torna al punto 3
5. Controlla se ha ancora byte rimanenti da codificare: se sì allora torna al punto 2, altrimenti la codifica termina

Di seguito il diagramma dell'algoritmo:

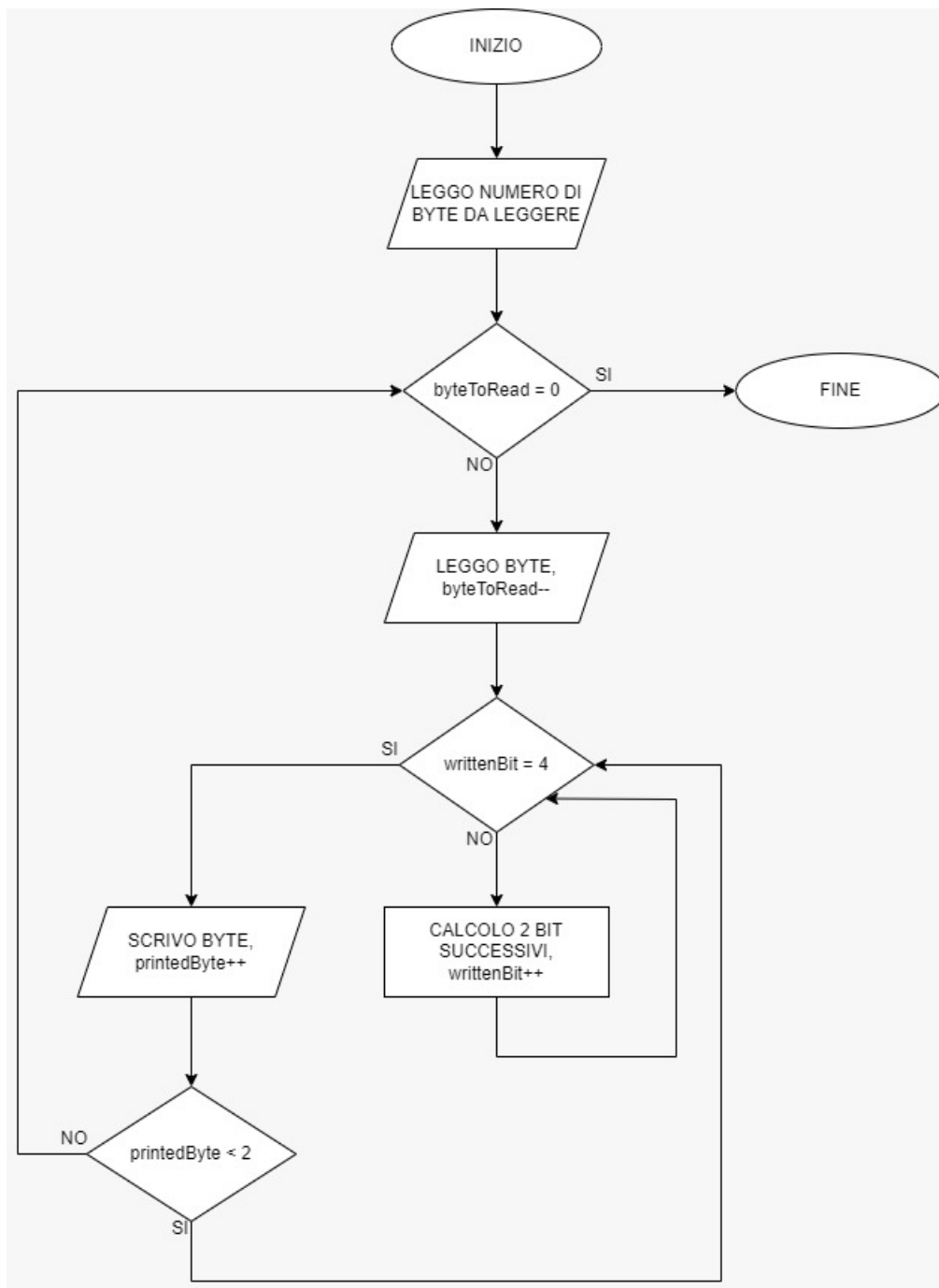


Figura 2: Algoritmo utilizzato

2.2 Macchina a Stati Finiti

L'algoritmo è quindi facilmente implementabile da una macchina a stati finiti, di cui segue la descrizione formale degli stati:

- **START**: Stato di idle in cui viene posta la FSM quando il segnale di reset è portato alto. Qui la macchina resetta i segnali usati nella computazione ai loro valori di default e attende che sia dato il via alla computazione quando il segnale `i.start` è portato alto. Quando ciò avviene porta a 1 anche `o.en` e setta `o.address` al primo indirizzo utile
- **WAIT_RAM**: Stato in cui riporto il segnale `o.en` basso e attendo l'aggiornamento della memoria
- **READ_W**: Stato in cui salvo in `byteToRead` il valore letto contenente i byte rimanenti da codificare
- **READ_BYTE_REQ**: Stato in cui se `byteToRead` è diverso da zero viene letto un nuovo byte dallo stream di input e decrementato di un'unità `byteToRead`, altrimenti la macchina è portata nello stato di **SET_DONE**
- **WAIT_READ**: Stato in cui aspetto l'aggiornamento dell'output della memoria
- **READ_BYTE**: Stato in cui salvo in `w.bitStream` il byte aggiornato proveniente dallo stream di input
- **EXECUTION**: Stato computazionale che si occupa di emulare il funzionamento del convolutore. Finché non ho prodotto un byte da stampare (ovvero finché non ho letto 4 bit da `w.bitStream`) torno in questo stato, altrimenti vado in **WRITE_BYTE**
- **WRITE_BYTE**: Stato che si occupa della stampa del byte prodotto. Se non ho finito di leggere tutti i byte di `w.bitStream` allora torno nuovamente in **EXECUTION**, altrimenti vado in **READ_BYTE_REQ** per leggere un nuovo byte o terminare la computazione
- **SET_DONE**: Stato in cui porto `o.done` alto
- **DONE_WAIT**: Stato in cui attendo che il segnale di start venga portato basso per poi tornare nello stato di **START** che riporta `o.done` a 0 ed è pronto per una nuova computazione

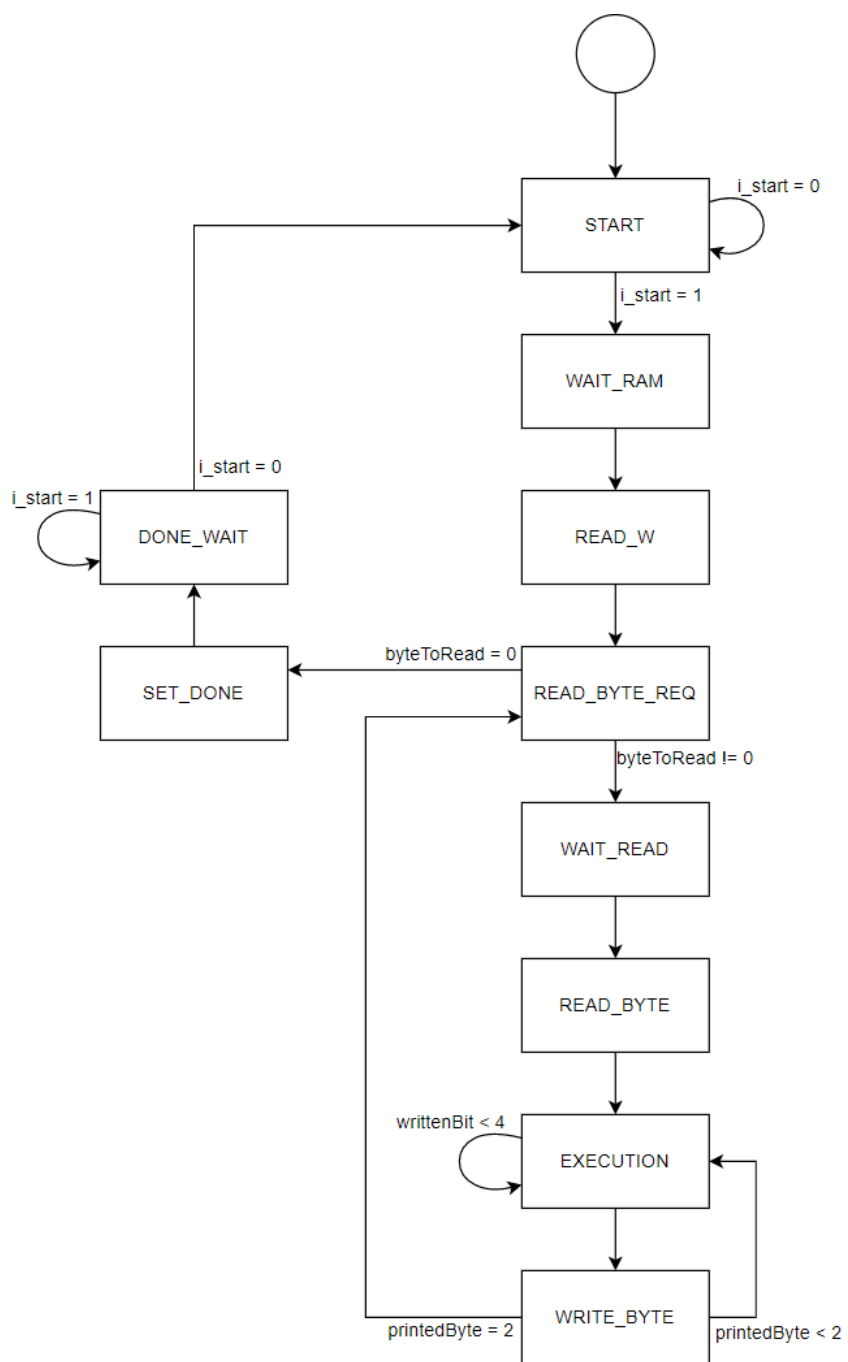


Figura 3: Rappresentazione Macchina a Stati Finiti

2.3 Schema dell'Implementazione

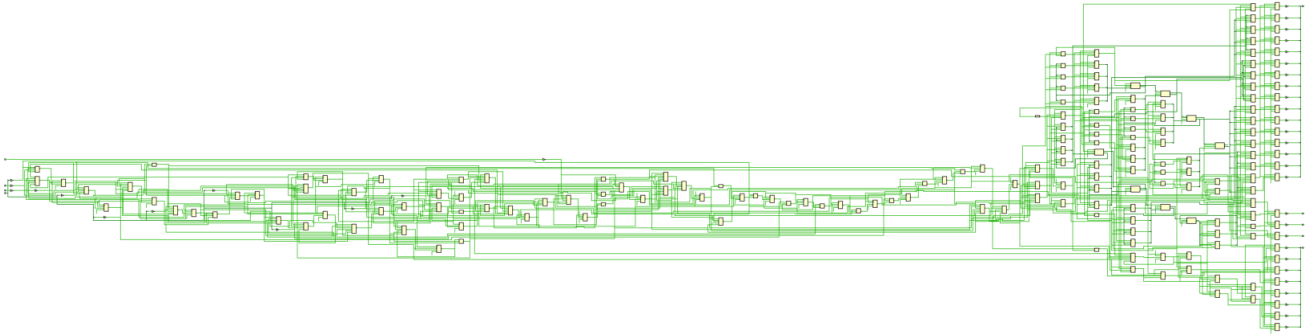


Figura 4: Schema dell'Implementazione

3 Risultati sperimentali

3.1 Testing

Conclusa la descrizione e la sintesi del componente si sono effettuati una serie di test per verificarne il corretto funzionamento.

- Oltre ai testbench d'esempio forniti si è creato un generatore casuale, con l'aiuto di uno script python, capace di raccogliere un'ampia casistica di test per poi sottoporli un elevato numero di volte, al fine di minimizzare la presenza di errori. I dati prodotti sono stati passati a due testbench che verificano il funzionamento del componente in presenza di reset, il primo, e in assenza di tale segnale, il secondo. Tutti i test sono stati passati nella loro interezza.
- Sono stati inoltre svolti test specifici per andare a coprire dei possibili corner case della specifica:
 - Andamento in presenza di sequenze in ingresso minime
 - Andamento in presenza di sequenze in ingresso massime
 - Memoria RAM vuota

3.2 Report di Sintesi

Per quanto concerne l'area utilizzata dal componente, la sintesi riporta il seguente utilizzo di componenti:

- LUT: 71

- FF: 93

In particolare si sottolinea l'assenza di latch nel componente realizzato. Analizzando l'aspetto temporale del componente valutato post sintesi si evince che il constraint imposto sul clock è rispettato: lo slack è di $95.850ns$ (come riportato in Figura 5); quindi i vincoli sono soddisfatti.

```
Timing Report

Slack (MET) :          95.850ns  (required time - arrival time)
Source:          writtenBit_reg[2]/C
                  (rising edge-triggered cell FDRE clocked by clock  (rise@0.000ns fall@50.000ns period=100.000ns))
```

Figura 5: Report Time Utilization

4 Note Conclusive

Il componente è stato realizzato rispettando le specifiche progettuali ed è stato testato in maniera estensiva casualmente e con testbench specifici per stimolare eventuali errori dovuti a corner case. Per quanto riguarda l'architettura del progetto, in principio si era progettata in maniera modulare con due process, uno che gestisse la parte relativa al clock, ovvero l'aggiornamento dei segnali ad ogni ciclo, e l'altro che gestisse invece i passi della FSM; successivamente si è deciso di mantenere un singolo process che si occupa di gestire entrambe le parti, a favore di una maggiore semplicità del codice e gestione dei segnali.